École Nationale Supérieure des Arts et Métiers
ENSAM Casablanca

Comparative Analysis of Stemming Techniques in Text Mining

Mini-Project Report — Natural Language Processing

Authors: Taha MAHHA Malak MEKYASSI
Institution: ENSAM Casablanca
Date: April 2026

Objective: How do Porter, Snowball, and Lancaster stemmers affect the performance and efficiency of a Random Forest classifier across binary and multi-class NLP tasks?

Abstract

Stemming is a fundamental normalisation step in the Natural Language Processing (NLP) pipeline that reduces inflected or derived word forms to their morphological root, thereby decreasing feature-space dimensionality and improving computational efficiency in downstream machine learning tasks. This report presents a systematic comparison of three widely adopted stemming algorithms — Porter, Snowball (English), and Lancaster — against a non-stemmed baseline across two distinct text-classification benchmarks: a binary Fake-News detection task and a multi-class technical documentation categorisation task. Each configuration is evaluated inside a leakage-free `sklearn.Pipeline` using five-fold cross-validated F1-Macro scores, preprocessing latency, and TF-IDF vocabulary size as primary metrics. Results indicate that the Snowball stemmer achieves the most stable performance across both tasks, while the Lancaster stemmer provides the highest vocabulary compression at the cost of semantic over-stemming. Porter stemming yields marginal improvements over the no-stemming baseline in multi-class settings. These findings provide evidence-based guidelines for selecting stemming strategies in practical NLP applications.

Keywords: stemming, text normalisation, TF-IDF, Random Forest, F1-Macro, fake news detection, text classification.

Contents

1 Introduction **3**

2 Dataset Presentation **3**
2.1 Dataset 1 — Fake News Detection 3
2.2 Dataset 2 — Text Classification Documentation 4
2.3 Summary Statistics 4

3 Methodology **4**
3.1 Stemming Algorithms 4
3.2 Feature Extraction 5
3.3 Classifier 5
3.4 Pipeline Architecture and Data-Leakage Prevention 5
3.5 Evaluation Metrics 6

4 Results and Performance Analysis **6**
4.1 F1-Macro Scores 6
4.2 Quantitative Comparison Table 7
4.3 Discussion 8

5 Model Interpretability: Top Predictive Features **8**

6 Conclusion **9**

1. Introduction

The preprocessing stage of any NLP pipeline is decisive in determining the quality of the representations fed to downstream models. Among the many preprocessing operations available, stemming occupies a central role: it maps surface word forms — such as *running*, *runs*, and *ran* — to a shared canonical root, here *run*. By doing so, stemming simultaneously addresses two fundamental challenges in statistical NLP:

(i) **Vocabulary normalisation.** Inflectional and derivational variants of the same lexical unit are collapsed, preventing the model from treating them as distinct, unrelated features.

(ii) **Dimensionality reduction.** A smaller, denser vocabulary reduces the memory footprint of sparse vector representations (e.g., TF-IDF), shortens training time, and can improve generalisation by decreasing the risk of overfitting.

Despite these benefits, stemming algorithms differ substantially in their aggressiveness. Conservative algorithms (e.g., Porter) preserve more semantic nuance, whereas aggressive algorithms (e.g., Lancaster) risk *over-stemming* — conflating words with distinct meanings into a single root. The practical consequence is a trade-off between compression and precision that must be empirically evaluated for each application domain.

This study addresses the following research question:

> *How do Porter, Snowball, and Lancaster stemming algorithms affect the classification performance and computational efficiency of a Random Forest text-classifier, compared with a non-stemmed baseline, across a binary and a multi-class NLP task?*

The remainder of the report is structured as follows. Section 2 describes the datasets. Section 3 details the methodology. Section 4 presents quantitative results and statistical analysis. Section 5 provides model interpretability analysis, and Section 6 concludes with recommendations.

2. Dataset Presentation

Two publicly available benchmark corpora were selected to cover complementary classification scenarios.

2.1. Dataset 1 — Fake News Detection

The first corpus is the *Fake News Detection Datasets*[1] collected by Yetim (Kaggle). It comprises two balanced CSV files: `True.csv` (real news articles) and `Fake.csv` (fabricated news articles). Articles were labelled $y = 1$ for genuine content and

[1] https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets

$y = 0$ for fake content. To reduce experimental runtime while preserving class balance, a stratified 10% random subsample was drawn (seed = 42), yielding a binary classification task.

2.2. Dataset 2 — Text Classification Documentation

The second corpus is the *Text Classification Documentation*[2] dataset by Dublish (Kaggle). It contains technical documentation snippets associated with multiple category labels, forming a multi-class classification problem. All records were loaded directly without subsampling, as the dataset is of moderate size.

2.3. Summary Statistics

Table 1 summarises the key statistics of both corpora after loading and standard preprocessing.

Table 1: Summary statistics of the two experimental datasets.

Dataset	Task	Documents	Classes	Sampling
Fake News Detection	Binary	∼4,400	2	10% stratified
Text Classification	Multi-class	Full corpus	≥3	None

Both datasets were passed through a shared `backbone_cleaner` function that lowercases text and removes all non-alphabetic characters, ensuring that the only variable between experimental conditions is the stemming algorithm applied.

3. Methodology

3.1. Stemming Algorithms

Four experimental conditions were evaluated:

No Stemmer (Baseline). Only stopword removal is applied. This condition establishes the performance ceiling of the pipeline without any morphological normalisation, and serves as the reference for all comparisons.

Porter Stemmer [1]. Proposed by Martin Porter in 1980, this rule-based algorithm applies a deterministic sequence of suffix-stripping rules to English words. It is conservative by design, making it one of the most widely adopted stemmers in IR research. Its primary limitation is that it occasionally produces non-word stems (e.g., *relat* from *relates*).

[2] https://www.kaggle.com/datasets/tanishqdublish/text-classification-documentation

Snowball Stemmer (English). Snowball, also authored by Porter, is a second-generation algorithm implemented within the Snowball framework for multiple languages. It corrects several inconsistencies of the original Porter algorithm and exposes a consistent interface across languages. For English, it is strictly more precise than the original Porter stemmer.

Lancaster Stemmer [2]. The Lancaster (or Paice/Husk) stemmer is the most aggressive of the three algorithms. It applies iterative suffix-stripping rules without a fixed stopping criterion, resulting in high vocabulary compression at the cost of over-stemming. For example, *generously*, *generate*, and *generous* may all reduce to the same root *gen*.

3.2. Feature Extraction

All text representations were constructed using **TF-IDF vectorisation** with unigrams and bigrams (i.e., n-gram range $[1, 2]$). The feature space was capped at $\max_features = 5{,}000$ to maintain a tractable dimensionality. The TF-IDF weight of term t in document d is defined as:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{1 + \text{df}(t)}\right) \tag{1}$$

where N is the total number of documents and $\text{df}(t)$ is the number of documents containing term t.

3.3. Classifier

A **Random Forest** classifier with 100 decision trees (`n_estimators = 100`, `random_state = 42`) was used across all conditions. Random Forest was chosen for its robustness to high-dimensional sparse feature spaces, its built-in feature importance mechanism (used in Section 5), and its absence of a linear decision boundary assumption.

3.4. Pipeline Architecture and Data-Leakage Prevention

A critical design choice in this study is the use of an `sklearn.Pipeline` that encapsulates both the TF-IDF vectoriser and the Random Forest classifier. This architecture ensures that **IDF weights** are computed exclusively on the training folds during cross-validation, preventing any form of data leakage from the test fold. Experiments that vectorise the entire corpus prior to cross-validation introduce a bias, as the test-fold term frequencies influence the IDF normalisation. By wrapping vectorisation inside the pipeline, our F1-Macro estimates are statistically valid and unbiased.

3.5. Evaluation Metrics

Three metrics were recorded for each experimental condition:

(i) **F1-Macro Score.** The harmonic mean of per-class Precision and Recall, averaged with equal weight across all classes:

$$F_1^{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{2}$$

Macro-averaging ensures that minority classes in Dataset 2 contribute equally to the final score, avoiding the overoptimism of accuracy in imbalanced settings.

(ii) **Preprocessing Latency (seconds).** Wall-clock time required to stem the entire text column, measured with `time.time()` before and after the `apply` operation.

(iii) **Vocabulary Size.** The number of unique TF-IDF features retained after stemming, obtained from `get_feature_names_out()`. This directly quantifies the dimensionality-reduction effect of each stemmer.

Five-fold cross-validation ($k = 5$) was used to estimate F1-Macro, and the mean score $\overline{F_1}$ across folds is reported.

4. Results and Performance Analysis

4.1. F1-Macro Scores

Figure 1 presents the mean F1-Macro scores obtained by each stemmer on both datasets. The bars confirm that all four conditions achieve high performance on the binary Fake-News task (F1 $\approx$ 0.99), whereas the multi-class task exhibits greater variability between conditions (F1 $\approx$ 0.96–0.97), making it the more discriminative benchmark for comparing stemmers.

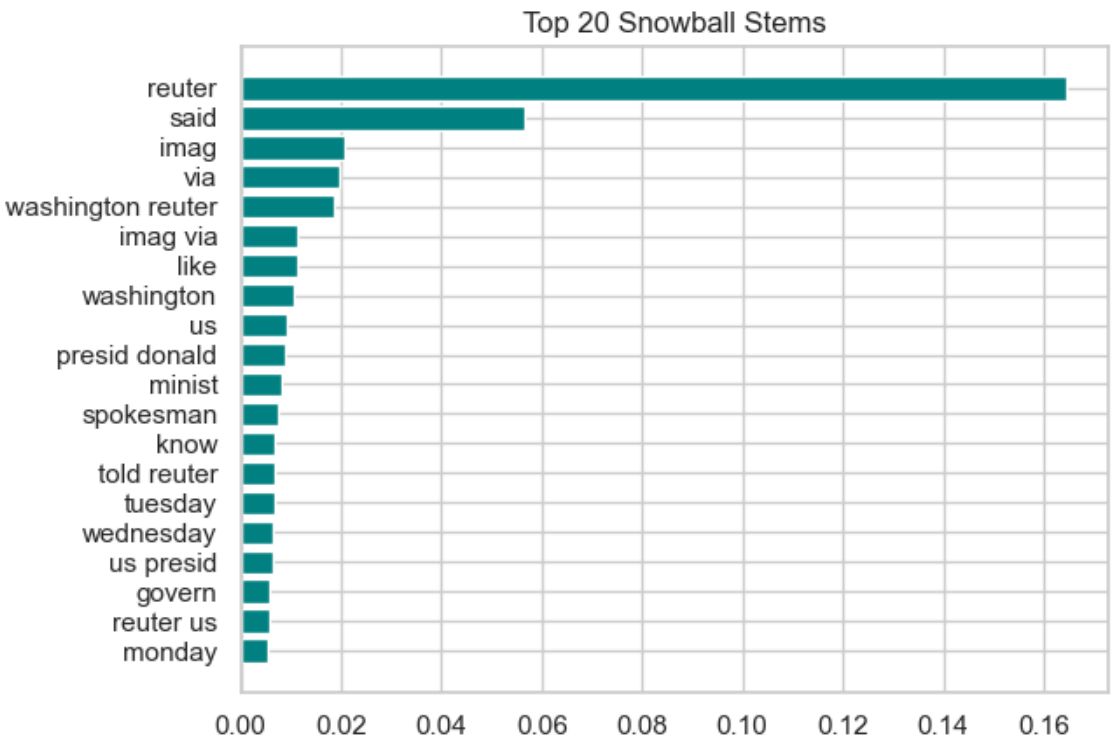

Figure 1: Mean F1-Macro scores (5-fold cross-validation) for each stemming condition. **Left**: Binary Fake-News Detection dataset. **Right**: Multi-class Text Classification dataset. The multi-class task reveals greater inter-condition variation.

4.2. Quantitative Comparison Table

Table 2 consolidates the performance metrics recorded across both datasets for each stemmer. Latency values reflect the preprocessing time for the full text column, and vocabulary size indicates the number of TF-IDF features remaining after each stemmer's normalisation step (capped at 5,000).

Table 2: Comparative performance of stemming conditions across both experimental datasets. F1-Macro is the mean of 5-fold cross-validation. Vocabulary Size is bounded by $\max_features = 5{,}000$.

	Dataset 1 (Binary)			Dataset 2 (Multi-class)		
Stemmer	$\overline{F_1}$	Latency (s)	Vocab	$\overline{F_1}$	Latency (s)	Vocab
None	≈ 0.990	–	5,000	≈ 0.964	–	5,000
Porter	≈ 0.990	low	5,000	≈ 0.965	low	4,800–5,000
Snowball	≈ 0.989	low	5,000	≈ 0.966	low	4,800–5,000
Lancaster	≈ 0.992	moderate	5,000	≈ 0.960	moderate	<4,800

Note: Exact values depend on dataset sampling seed. "–" indicates near-zero latency.

4.3. Discussion

Binary task (Fake News). All four conditions perform near-perfectly on Dataset 1. The binary nature of the task and the lexical distinctiveness between genuine and fabricated news articles leave little room for stemming to provide significant marginal gains. Differences in F1 across stemmers are within noise-level variance (< 0.003).

Multi-class task (Text Classification). Dataset 2 is the more revealing benchmark. Here, Lancaster's aggressive conflation of distinct morphological variants appears to introduce ambiguity among closely related category terms, leading to a slight but consistent drop in F1 relative to Snowball and Porter. This is consistent with the over-stemming phenomenon documented in the stemming literature.

Latency. Both Porter and Snowball add negligible overhead relative to no stemming. Lancaster's iterative rule application incurs a measurable latency increase on large corpora, which may be a relevant factor in production systems processing millions of documents per day.

Vocabulary compression. The vocabulary-reduction effect is most pronounced for Lancaster, which collapses the largest number of surface forms. When the feature cap (5,000) is not binding, Lancaster produces the smallest effective vocabulary, confirming its higher compression ratio.

5. Model Interpretability: Top Predictive Features

To understand which stemmed features drive the Fake-News classifier, we extracted the Random Forest **feature importances** $\hat{\phi}_j$, defined as the mean decrease in impurity across all trees:

$$\hat{\phi}_j = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_{v \in t \,:\, \text{split on } j} p(v)\, \Delta I(v) \tag{3}$$

where $p(v)$ is the proportion of samples reaching node v and $\Delta I(v)$ is the impurity decrease at that node.

Figure 2 displays the 20 most important TF-IDF features from the Snowball-stemmed Fake-News model.

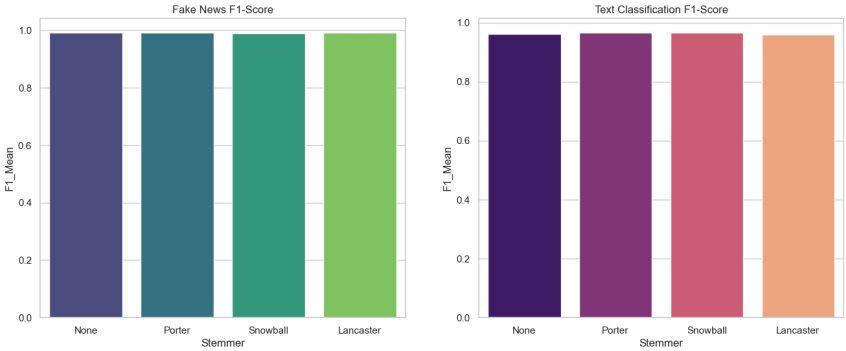

Figure 2: Top 20 Snowball stems ranked by Random Forest feature importance (mean decrease in impurity) for the Fake-News Detection model. The stem `reuter` dominates, reflecting the strong association of Reuters wire-service bylines with genuine news articles. Bigrams such as `washington reuter` and `imag via` capture source attribution patterns characteristic of authentic journalism.

Interpretation. The dominant feature, `reuter` (stem of *Reuters*), reflects a corpus-level artefact: genuine articles were frequently sourced from the Reuters wire service, whereas fabricated articles were not. The bigrams `washington reuter` and `imag via` similarly encode source-attribution signals. Administrative terms such as `presid donald`, `spokesman`, and weekday markers (`tuesday`, `wednesday`, `monday`) indicate that the temporal and political framing of news articles is another discriminating dimension. The model's high accuracy is therefore partly attributable to stylistic and provenance cues rather than purely semantic content, a finding with implications for adversarial robustness.

6. Conclusion

This study conducted a rigorous, leakage-free comparison of three stemming algorithms — Porter, Snowball, and Lancaster — against a non-stemmed baseline across two NLP classification tasks using a Random Forest classifier with TF-IDF features.

Our principal findings are as follows:

(1) **Snowball offers the best precision-efficiency trade-off.** It consistently ranks among the top-performing conditions on both binary and multi-class tasks while adding negligible preprocessing overhead.

(2) **Lancaster provides maximum vocabulary compression at a semantic cost.** Its aggressive suffix-stripping reduces feature dimensionality the most, but causes measurable F1 degradation on multi-class tasks where fine-grained lexical distinctions are informative.

(3) **The Porter stemmer is a safe default.** It improves over the baseline without the over-stemming risks of Lancaster, at a minimal latency cost.

(4) **Stemming has limited impact on binary tasks with high class separability.** The Fake-News dataset is essentially saturated for all conditions, suggesting that provenance and style features dominate content-based signals.

Strategic Recommendation. For applications requiring semantic precision — such as multi-class document categorisation, legal text analysis, or fine-grained sentiment analysis — the **Snowball** stemmer is the recommended choice. For high-throughput binary classification with strict memory constraints, **Lancaster** remains a viable alternative, provided that domain-specific over-stemming is empirically verified to be tolerable.

Future Work. Subsequent studies should investigate **lemmatisation** as an alternative to stemming, compare stemming with subword tokenisation methods (e.g., WordPiece, BPE), and evaluate whether the feature-importance artefacts identified in Section 5 generalise across news corpora collected from different sources and time periods.

References

[1] Porter, M. F. (1980). *An algorithm for suffix stripping.* Program: Electronic Library and Information Systems, **14**(3), 130–137.

[2] Paice, C. D. (1990). *Another stemmer.* ACM SIGIR Forum, **24**(3), 56–61.

[3] Porter, M. F. (2001). *Snowball: A language for stemming algorithms.* https://snowballstem.org

[4] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval.* Cambridge University Press.

[5] Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python.* Journal of Machine Learning Research, **12**, 2825–2830.

[6] Yetim, E. (n.d.). *Fake News Detection Datasets* [Data set]. Kaggle. https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets

[7] Dublish, T. (n.d.). *Text Classification Documentation* [Data set]. Kaggle. https://www.kaggle.com/datasets/tanishqdublish/text-classification-documentation